

EASL System Maintenance Guide

1. Purpose and Scope

The purpose of this document is to define how the Ctrl-BIAS EASL pipeline should be maintained as a trustworthy, reproducible system across different iterations and team members. Its purpose is to ensure that every contributor, new or existing can execute the next cycle without uncertainty or rework.

The focus of this document is on the practical elements most prone to errors such as Python dependencies, annotation workflows (MTurk and Excel), data validation or embedding generation. Every guideline in this document is grounded in direct experience and each recommendation reflects something implemented, verified, or corrected in past runs.

The scope covers the complete EASL loop initialise, create, mark and refresh across the four foundation domains: Bloc Tilt, Worldview, Power Distance and Gender in Occupation. It also includes recovery procedures for failed or degraded runs: how to roll back safely, where to begin diagnosis and how to adjust cadence rather than forcing continuation of a broken process.

By following these steps, teams will maintain the reproducibility of results and preserve the validity of all subsequent analyses.

2. System Overview

The EASL (Efficient Annotation of Scalar Labels) system is the computational kernel of the Ctrl-BIAS project. The EASL system works by delivering a simplified workflow for dataset preparation, annotation and statistical updates with the aim to quantify bias in large language model (LLM) outputs.

The EASL's infrastructure includes several interlinked components. The Python scripts `initialize.py`, `main.py`, are designed to take care of the creation and maintenance of annotated data throughout the process. `main.py` operates using both "generate" and "update" arguments, creating Human Intelligence Task (HIT) files to annotate and then folding human judgments into the main dataset.

Annotation is performed principally by Amazon Mechanical Turk (MTurk) through customised HTML templates designed for bias rating tasks. In the event of MTurk unavailability, a side by side Excel based annotation format facilitates manual scoring. EASL code relies on BERT for producing semantic representations of each output. They are stored as JSON arrays inside the main Comma Separated Value (CSV) document and are used to group similar outputs within each HIT file to ensure that annotators see thematically coherent content.

3. Maintenance Responsibilities

We split responsibilities so that each part of the system has a clear owner. The technical lead or data engineer takes care of the Python environment and keeps `requirements.txt` pinned and healthy. After any upgrade, they run a short smoke test initialising a simple CSV, generating a single HIT file and opening the MTurk template in a browser to catch dependency issues early.

The annotation coordinator keeps the MTurk HTML and the Excel workbook aligned with the HIT schema. They ensure the instructions are plain, the examples are relevant and the 1–5 rating scale is obvious. They also monitor throughput if a batch is lagging, they adjust size or timing rather than letting a cycle stall.

Another role of the data engineer is to look after the correct implementation of word embeddings. Their job is to ensure the data in the `embedding` column stays valid (proper JSON arrays, no NaNs), with the aim to record the model and settings each time and to decide when a refresh is warranted. If we ever change model families (for example, from BERT to SBERT), they plan and execute a full regeneration so similarity is comparable across rows.

Finally, the data engineer validates every merged dataset and raises a flag if the validator finds systematic issues, missing identifiers, mismatched IDs, or coverage gaps where a prompt didn't receive all three models. Everyone contributes to light documentation: a few-line note for any change that might affect the next person who touches the pipeline.

4. Routine Maintenance Tasks

4.1 Environment and Dependencies

Having a standard Python environment prevents surprise script crashes. For every annotation or merge operation, virtual environment activation, dependency update and all tests have to be executed in advance. This will ensure local development team shared environments are consistent.

On environment activation, the dependencies need to be re-installed via the `requirements.txt` file, tailored for each environment after which a test run of the EASL pipeline on a minor dataset is done. This is to double-check if any recent library updates have accidentally corrupted primary functionality, particularly for embedding generation or data manipulation.

4.2 Data Integrity Checks

After each dataset merge or update, the maintainer should perform a full data validation process. The responsible script for this task is `easl_validate.py` which is used to identify any structural or logical inconsistencies, including missing identifiers, null annotations and mismatched model outputs.

The generated files should be reviewed by a second person for consistency. Any missing values such as prompt id, invalid score, or incomplete model coverage must be corrected before proceeding to the next annotation iteration.

Before running updates or merges, each version of the dataset should be backed up in the `/data/backups/` directory with a timestamp. These backups make it possible to restore the system quickly if a later merge introduces inconsistencies.

4.3 Annotation Pipeline: MTurk and Excel Fallback

MTurk path.

Test the HTML in Sandbox, even for small edits. Confirm the 1–5 scale is prominent, the guidance matches the discipline and the examples are brief but concrete. After launch, don't wait until the very end to export; pull a mid-run sample, skim it for obvious issues and adjust if needed. When the batch completes, export the results to a dated folder. Those files are the definitive record of that round.

Excel fallback (when MTurk is down).

The Excel workflow mirrors our HIT CSVs so the update step needs no code changes.

Generate as usual:

```
python main.py --operation generate --model <initialized_file.csv> --hits 20
```

Open *_hit_<n>.csv and copy the content. On the side also open the Master_Worksheet.xlsx. Paste the content of the HIT into the yellow tab “raw_hit” which then automatically generated data required for annotators to manually score each HIT in different tabs marked as HIT_# and denoted with the colour green. Every cell in this worksheet is locked for users except the tab raw_hit and the cells needed for ratings to prevent accidental override.

Raters move row by row and enter a single rating per output. When a workbook is complete, export the content of the red tab “result” to CSV with the same headers. Ensure to keep all rows and let the update consume multiple judgments. Name the consolidated file as if it were an MTurk export—discipline_raw_result_<n>.csv—and place it where the pipeline expects. Fold back with:

```
python main.py --operation update --model <initialized_file.csv>
```

If the update complains about missing annotations, check completeness against the HIT file. If it warns about mismatched IDs, restore the original identifiers in Excel and re-export.

4.4 Embedding Generation

Ensuring integrity is critical when clustering semantically equivalent outputs. At every instance of embeddings being created or updated, the maintainer needs to ensure that each row contains a valid JSON array and no parsing errors occur.

Depending on the capacity and the embedding objectives, the process of generation needs to be started with the following CLI prompt:

```
python main.py --operation generate --model worldview_1.csv --hits 25 --emb-weight 0.6
```

Embeddings distributions need to be regularly verified to ensure reliability from one run to another. In the case if the base embedding model is altered (for example, replacing BERT with ALBERT or with a domain-specific variant), all earlier values generated for embeddings needs to be recalculated to maintain data integrity.

4.5 Versioning and Backup

It is important to keep a consistent naming conventions for traceability. Each dataset iteration follows a specific pattern as denoted in the “Code base” submission and this format needs to be preserved thorough the process for EASL codes to function as intended. Before executing the next update command, the current dataset should be copied so a backup copy will be preserved. Each backup file must be named in a clear manner for traceability, such as:

gender_raw_1_backup.csv

A short log file or spreadsheet would maintain a record of which result file was merged into which dataset version.

5. Error Recovery Procedures

The table below outlines common failure or errors along with the recommended resolution.

Issue	Symptom	Resolution
CSV read/write error	Error message indicating a file cannot be read or written	Check file paths and encoding. Ensure the destination directory exists and the file is not open elsewhere.
Missing annotation columns	Validator flags every row as missing annotation	Confirm that the MTurk or Excel results include the correct headers.
Mismatched identifiers	Same output ID mapped to multiple models	Identify duplicates in the dataset and correct them before merging.
Embedding parse error	JSON decoding failure	Regenerate embeddings for affected rows and re-run validation.
MTurk upload failure	HITs remain unlisted or incomplete	Test the HTML template in Sandbox and re-upload the batch.
Unexpected annotation distribution	All scores cluster near extremes	Review annotator instructions and quality control questions for clarity.

6. Periodic Review Schedule

Regular maintenance ensures that the framework would continue to produce reliable data ready for analysis. The following review schedule proposes the ideal review frequency; however, the regular review is also dependant on the usage and the user's capacity.

Frequency	Task	Description
Weekly	Validate all new datasets	Run the EASL validator on each new or modified CSV file.
Monthly	Environment verification	Confirm that Python dependencies, scripts and virtual environments remain functional.
Quarterly	Embedding refresh	Recompute or verify embeddings to ensure consistency with the current BERT model.
Per semester	Documentation audit	Review the README and this maintenance guide for accuracy and completeness.

7. Security and Access Control

Maintaining the security and controlled access are essential to protecting data integrity.

Only authorized users should have write access to

the `/data/master` and `/data/qa` directories.

All AWS credentials, including MTurk login details, must be stored securely, preferably in an encrypted credential manager or memorised by the users. Annotator's data must never include personal or identifiable information. Access keys and permissions must be rotated on a regular basis and GitHub repositories must have branch protection enabled for critical scripts and dataset directories.

The team should also frequently scan the repository for public disclosure of credentials or public availability of private annotation data in error.

8. Logging and Monitoring

Comprehensive logging is one of EASL pipeline's strength and it provides visibility over every step of the process. Every major step from data ingestion and annotation through to validation and analysis we have a clear auditable record of what was run, by whom and under which configuration. As each step of the annotation provides a separate clearly labelled CSV file, it would act as an open trail that allows the team to monitor how each dataset, embedding, or model output was produced.

All files should be stored within a special directory of the repository and updated each time an important workflow completes. Looking at historical data, we can capture significant context such as parameter changes and noteworthy results.

9. Change Management

All modifications to the EASL codebase, datasets, or annotation pipelines must always be well-documented and transparent. The objective is to have consistency in the system, prevent performance degradation and make sure any changes however minimal is deliberate and reproducible.

All modifications need to be developed and tested in isolation before integrating them into the main codebase. It involves creating a separate Git branch for each modification while performing internal testing and passing it through a second developer's review before approval.

All changes that are accepted needs to be properly documented with the reason why it is made, what it affects and the expected impact. More significant updates, such as changing data schemas or annotation rules also needs to be reported to the broader team to ensure that everyone is on the same page.

By adhering to these practices on a continuous basis the project can evolve in a controlled and reproducible fashion. This approach keeps the integrity of the EASL pipeline developed by Ctrl-BIAS team intact while it ensures that any previous iteration of the system can be reproduced as needed.

Summary

This guide sets standards that keep the Ctrl-BIAS EASL pipeline predictable, transparent and scientifically sound. Through these upkeep procedures, the team has consistent data quality, protects system integrity and allows flexible annotation workflows in both MTurk as well as manual workflows.

A well-maintained EASL system generates results that are true to actual model behavior, comparable and consistent. Maintenance is more than systematic maintenance, it is the project's credibility itself and the credibility of its research findings' mark of authenticity.